Deep Learning (23AID211)

Convolutional Neural Networks (CNNs)

By

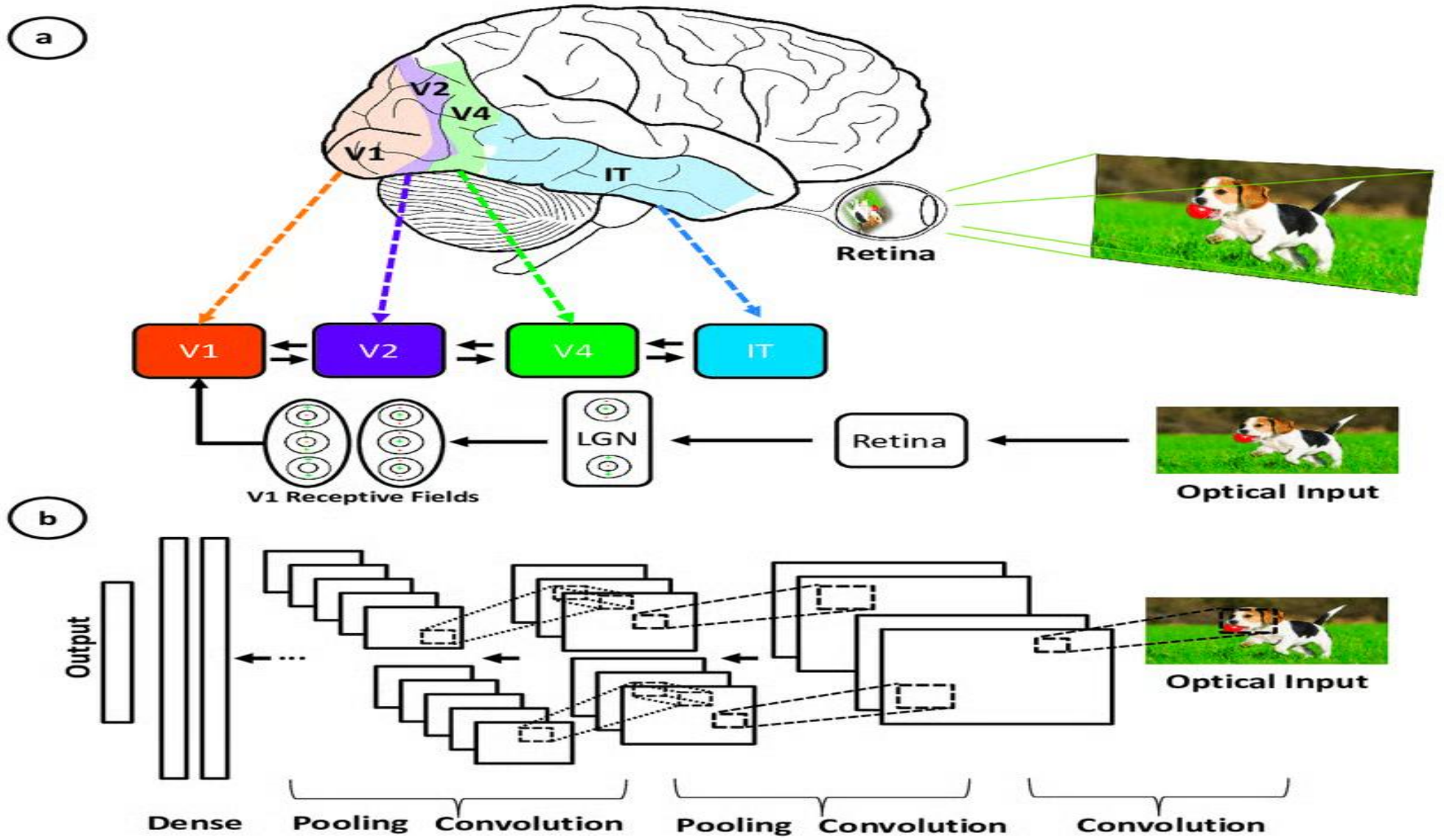
Dr. Vivek Patel

(B.Tech, M.Tech, Ph.D)

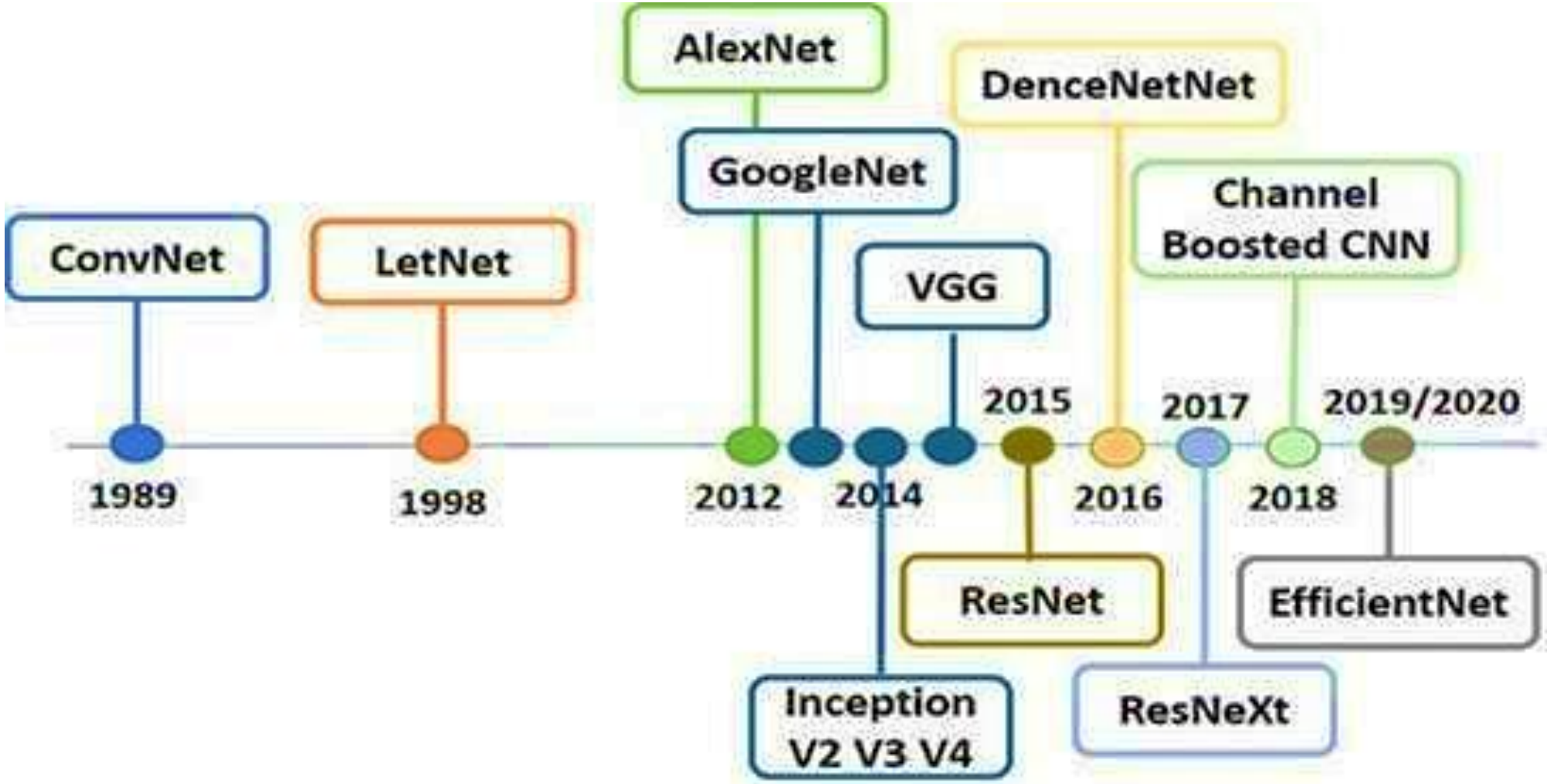
School of Artificial Intelligence, Amrita Vishwa Vidyapeetham

Faridabad Campus, Delhi NCR, India

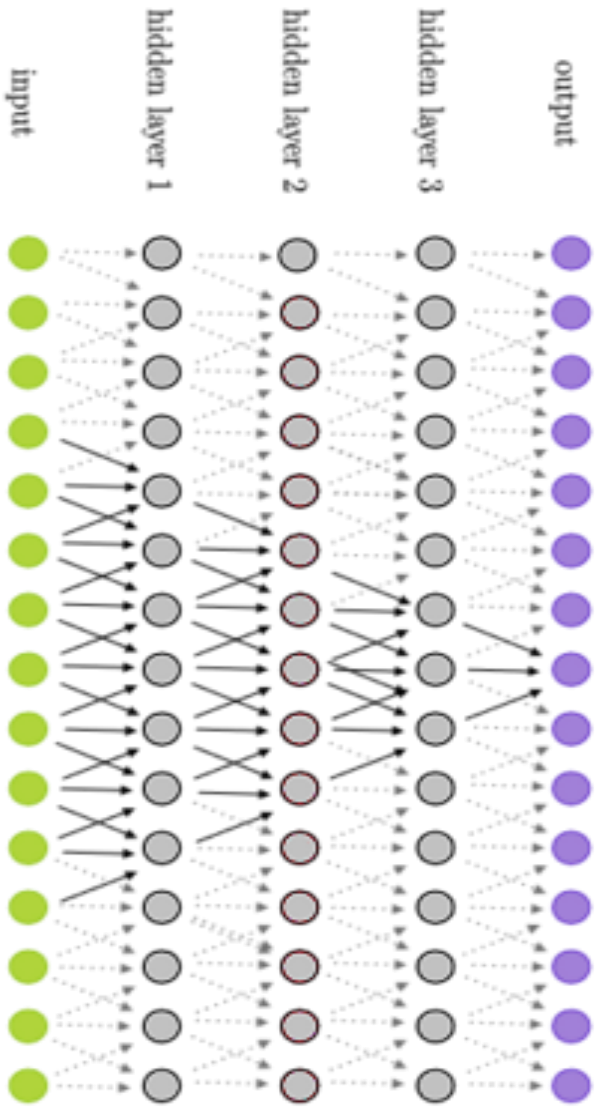
Convolutional Neural Networks



Convolutional Neural Networks



CNNs



1D convolutional neural network

LeNet

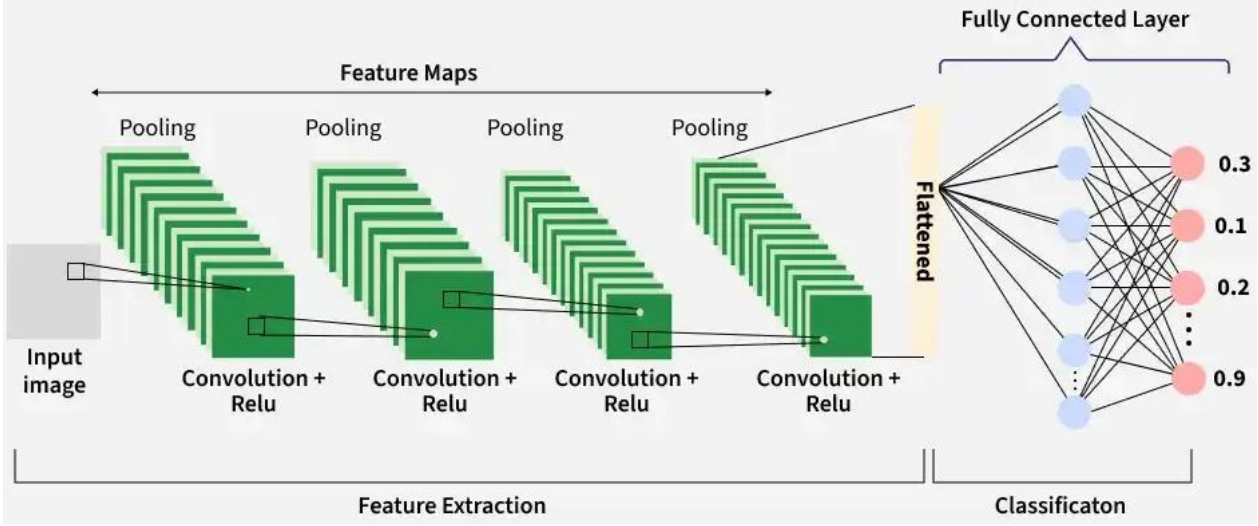
Image: 28 (height) × 28 (width) × 1 (channel)
↓
Convolution with 5×5 kernel+2padding:28×28×6
↓ sigmoid
Pool with 2×2 average kernel+2 stride:14×14×6
↓
Convolution with 5×5 kernel (no pad):10×10×16
↓ sigmoid
Pool with 2×2 average kernel+2 stride: 5×5×16
↓ flatten
Dense: 120 fully connected neurons
↓ sigmoid
Dense: 84 fully connected neurons
↓ sigmoid
Dense: 10 fully connected neurons
↓
Output: 1 of 10 classes

AlexNet

Image: 224 (height) × 224 (width) × 3 (channels)
↓
Convolution with 11×11 kernel+4stride:54×54×96
↓ ReLu
Pool with 3×3 max. kernel+2 stride: 26×26×96
↓
Convolution with 5×5 kernel+2 pad:26×26×256
↓ ReLu
Pool with 3×3 max.kernel+2stride:12×12×256
↓
Convolution with 3×3 kernel+1 pad:12×12×384
↓ ReLu
Convolution with 3×3 kernel+1 pad:12×12×384
↓ ReLu
Convolution with 3×3 kernel+1 pad:12×12×256
↓ ReLu
Pool with 3×3 max.kernel+2stride:5×5×256
↓ flatten
Dense: 4096 fully connected neurons
↓ ReLu, dropout p=0.5
Dense: 4096 fully connected neurons
↓ ReLu, dropout p=0.5
Dense: 1000 fully connected neurons
↓
Output: 1 of 1000 classes

CNN Architecture

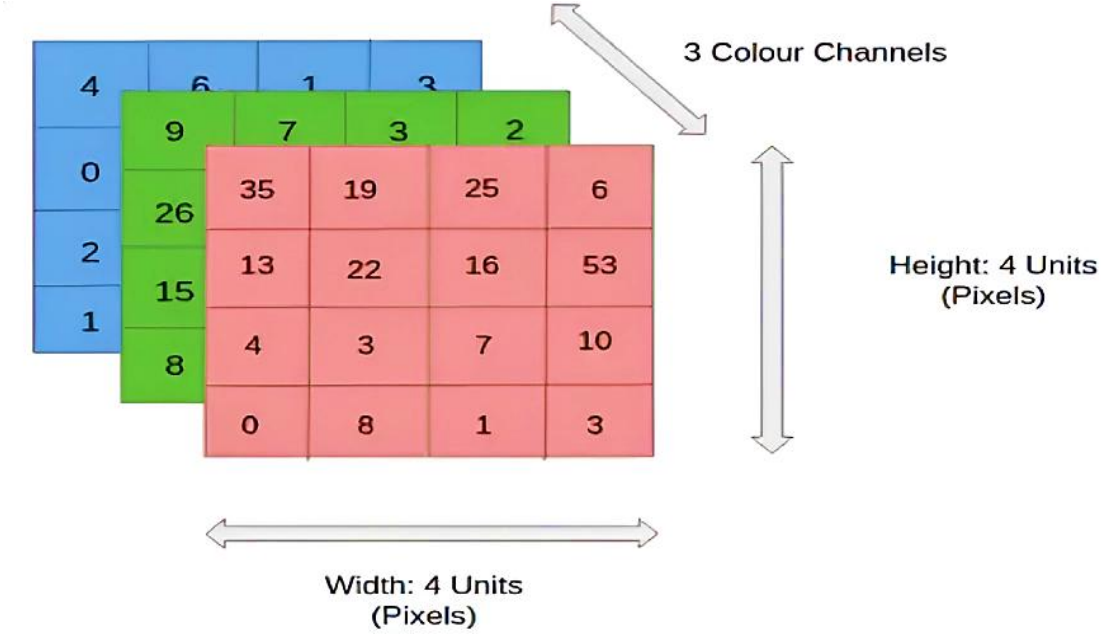
- **Convolutional Neural Networks (CNNs),** also known as ConvNets, are neural network architectures inspired by the human visual system and are widely used in computer vision tasks For capturing spatial relationships between pixels.



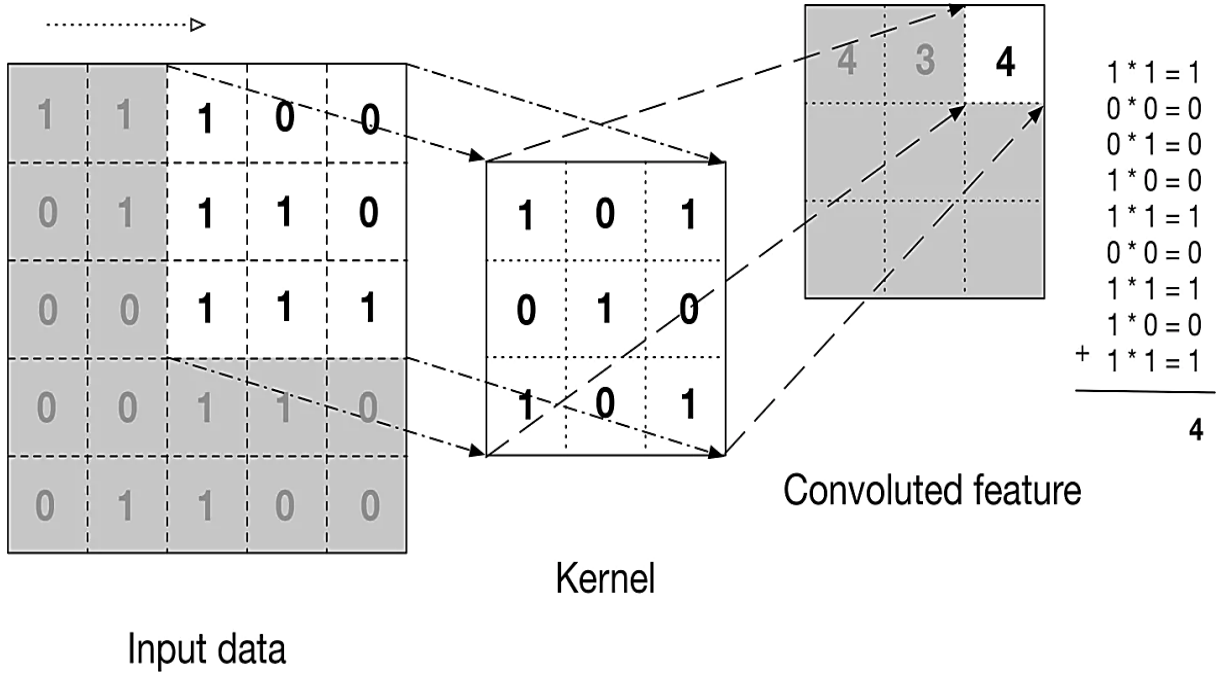
- Automatically learn hierarchical features through convolution operations, from simple edges and textures to complex shapes and objects.
- Detect objects at different positions within an image, ensuring robustness to spatial variations.
- Reduce computational complexity by processing local regions instead of the entire image at once.

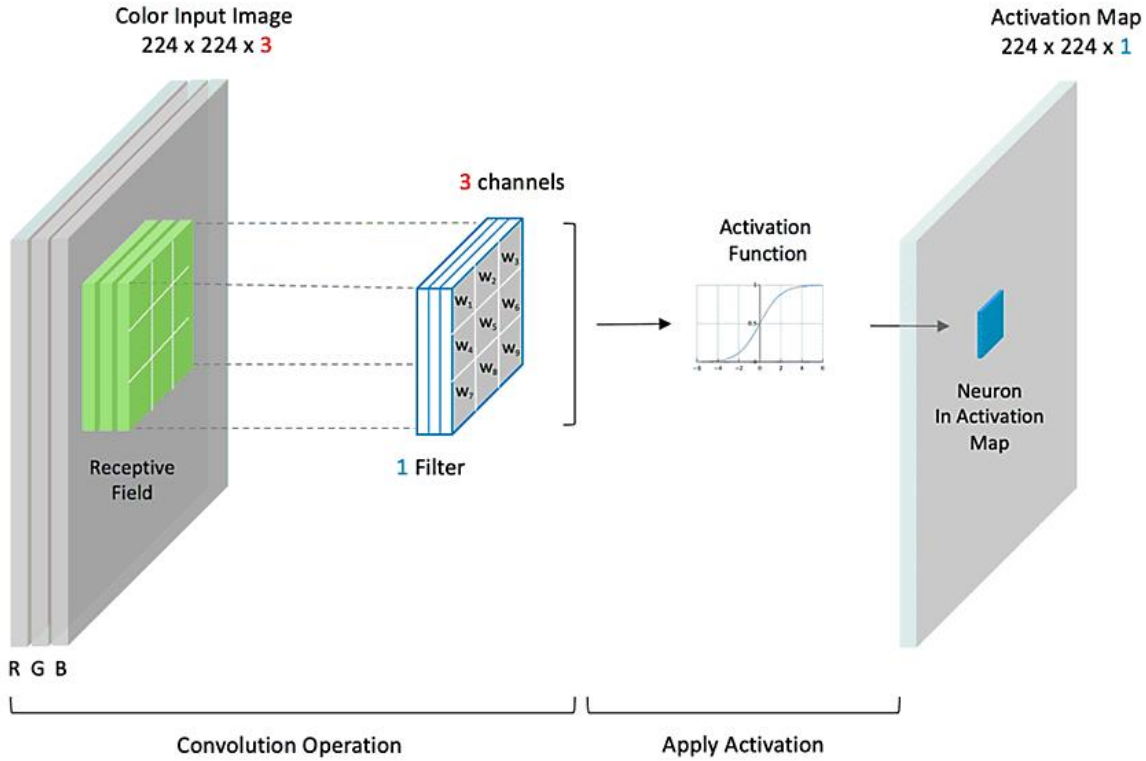
Key Components of CNN

- **Input Layer:** The input layer receives the raw image data and passes it to the network for processing. In CNNs, input is typically a 3D volume (width × height × depth).



Convolutional Layer: The Convolutional Layer is responsible for extracting important features from the input data. It applies a set of learnable filters (kernels) that slide over the image and compute the dot product between the filter weights and corresponding image patches, producing feature maps.



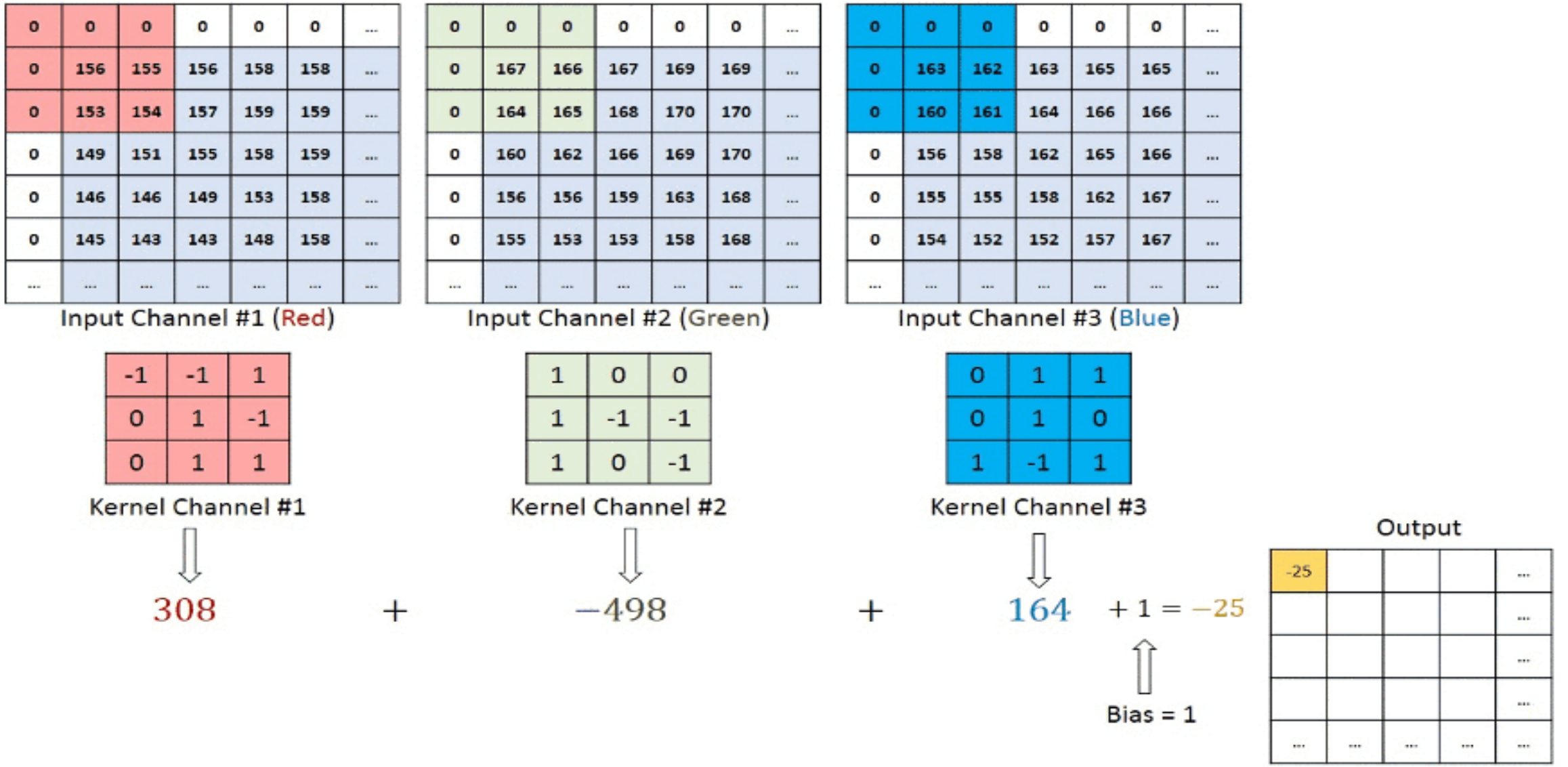


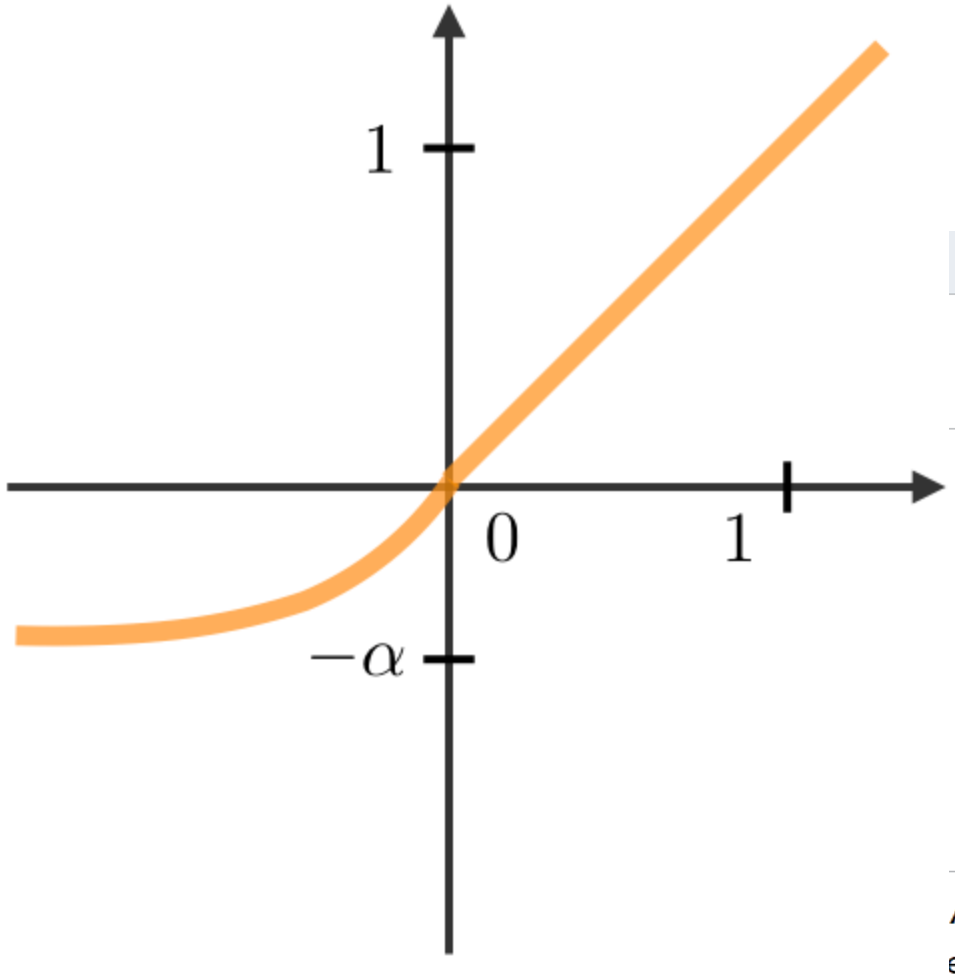
A 2D Convolution output feature size is determined by the formula, where,

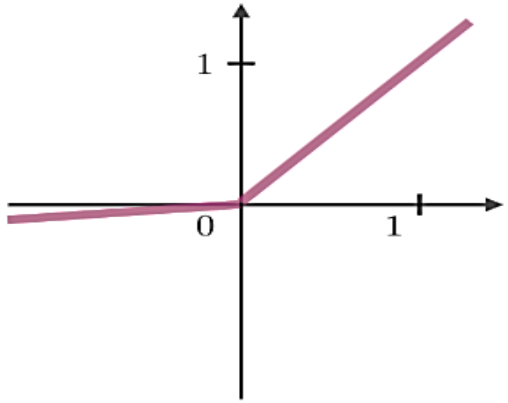
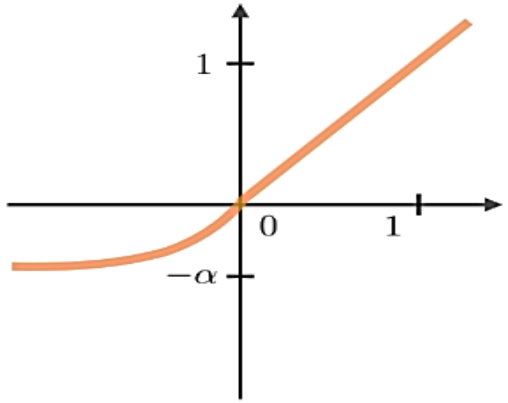
$$O = \left\lfloor \frac{n - f + 2p}{s} \right\rfloor + 1$$

- O: Output size
- n: Input size (height or width)
- f: kernel size
- p: Padding
- s: Stride

Convolutional Operation



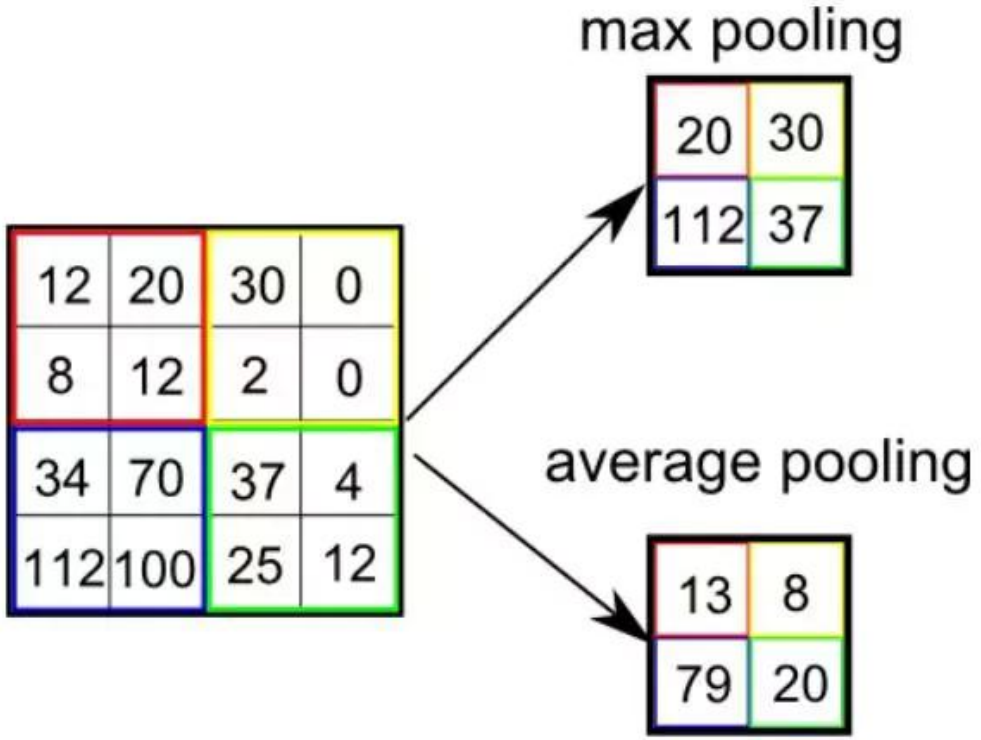


Leaky ReLU	ELU
$g(z) = \max(\epsilon z, z)$ with $\epsilon \ll 1$	$g(z) = \max(\alpha(e^z - 1), z)$ with $\alpha \ll 1$
	
Addresses dying ReLU issue for egative values	• Differentiable everywhere

The Activation Layer introduces non-linearity into the network by applying an element-wise activation function to the output of the convolution layer. This enables the model to learn complex patterns beyond linear relationships.

CNN Architecture

Pooling Layer: The Pooling Layer is used to reduce the spatial dimensions of the feature maps, making computation faster, reducing memory usage and helping to prevent overfitting. It is typically inserted between convolutional layers in a CNN.

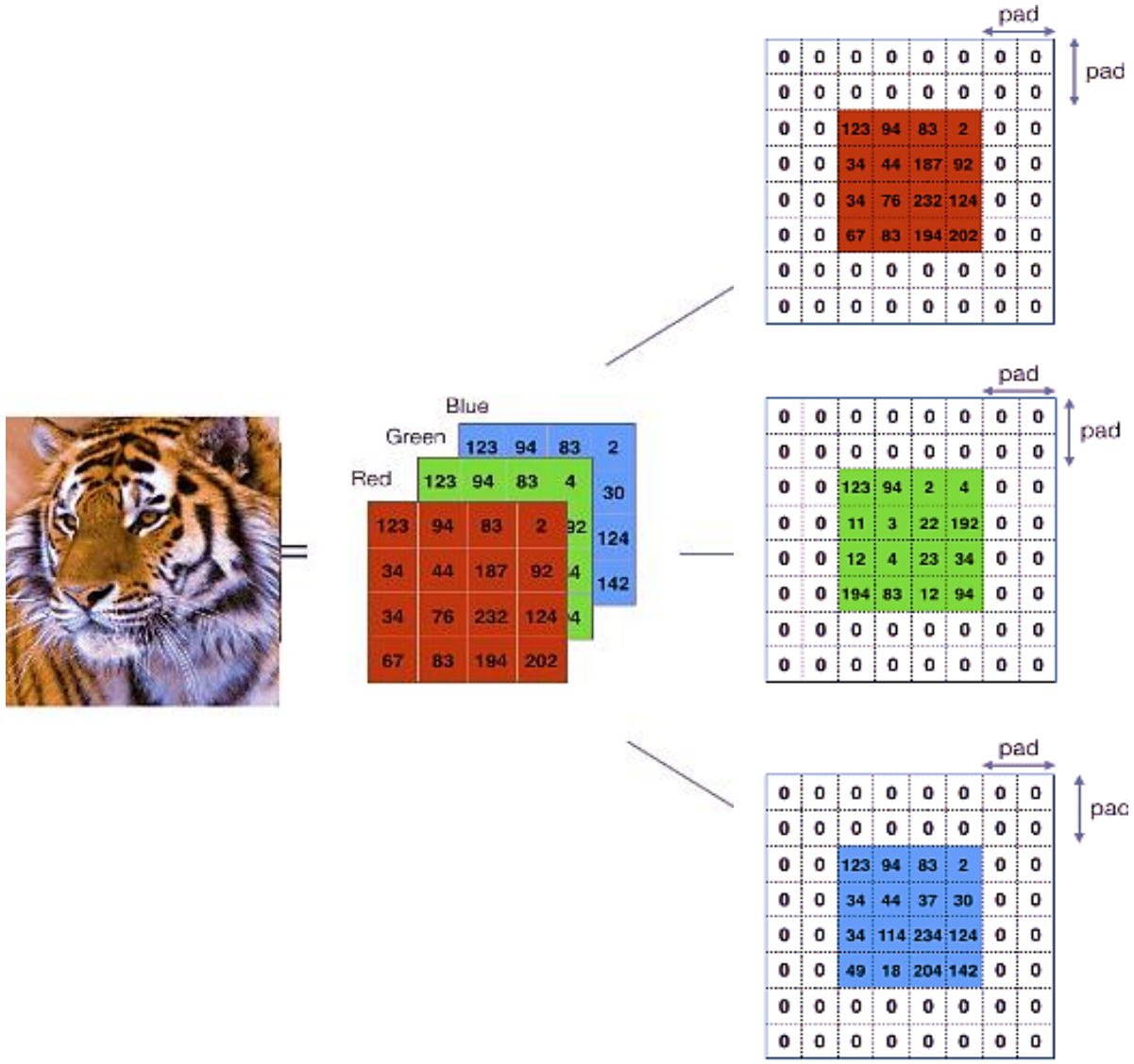




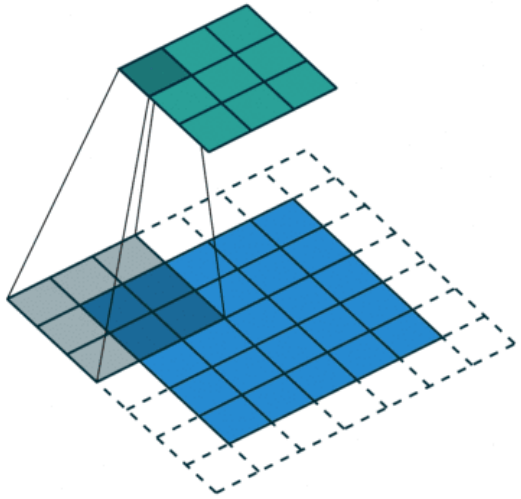


Padding

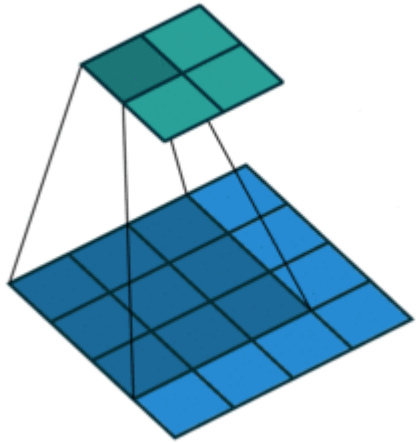
- **Padding** is an important parameter in CNN, which helps to preserve the input spatial dimension by adding extra pixels around the input image borders.
- By conserving border information, helps to improve model performance in determining the output spatial size of feature maps.



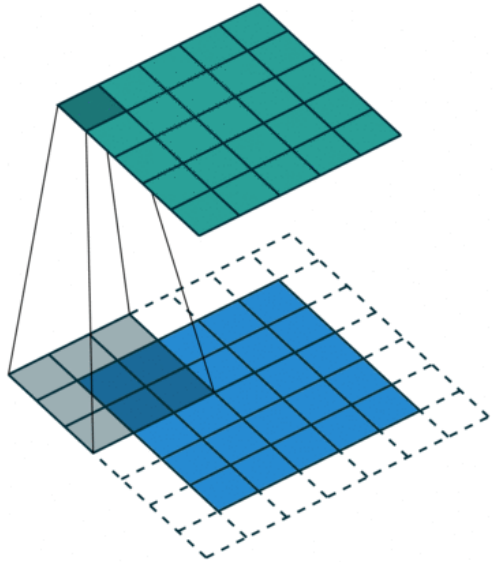
Padding



With Padding Stride = 2



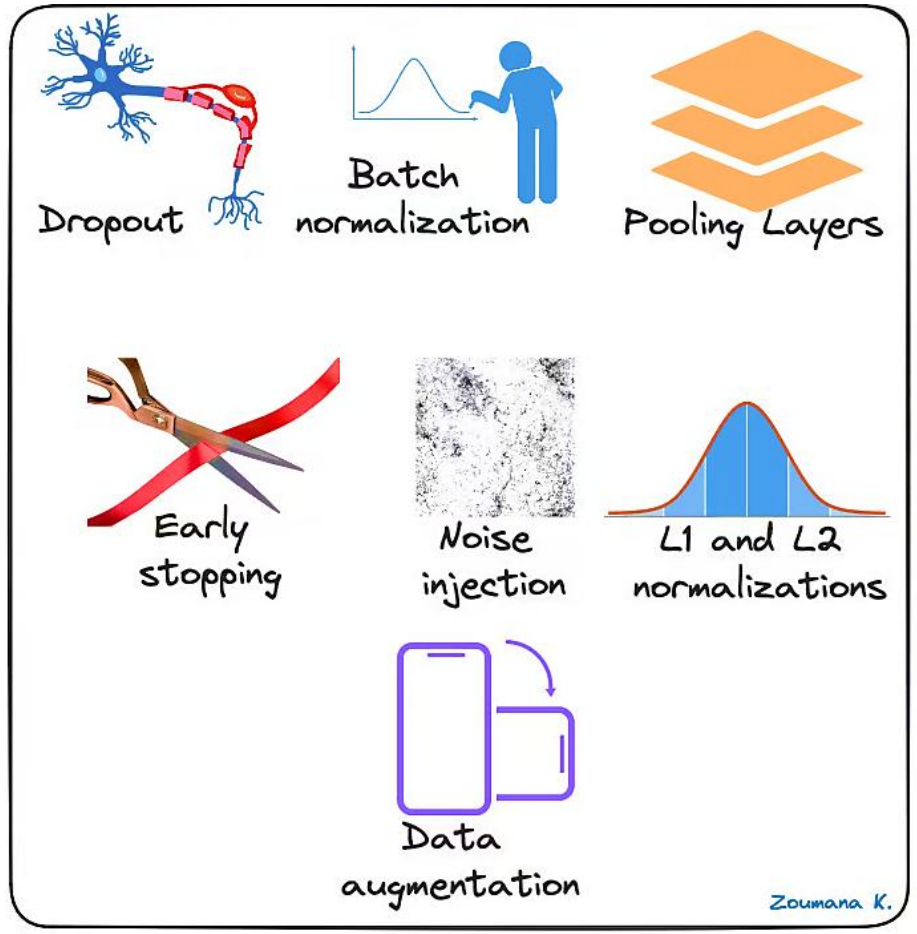
No Padding Stride = 1



Same Padding Stride =1

Strategies to Mitigate Overfitting in CNNs

- **Dropout:** This consists of randomly dropping some neurons during the training process, which forces the remaining neurons to learn new features from the input data.
- **Batch normalization:** The overfitting is reduced at some extent by normalizing the input layer by adjusting and scaling the activations. This approach is also used to speed up and stabilize the training process.
- **Pooling Layers:** This can be used to reduce the spatial dimensions of the input image to provide the model with an abstracted form of representation, hence reducing the chance of overfitting.



- **Early stopping:** This consists of consistently monitoring the model's performance on validation data during the training process and stopping the training whenever the validation error does not improve anymore.

Strategies to Mitigate Overfitting in CNNs

- **Noise injection:** This process consists of adding noise to the inputs or the outputs of hidden layers during the training to make the model more robust and prevent it from a weak generalization.
- **L1 and L2 normalizations:** Both L1 and L2 are used to add a penalty to the loss function based on the size of weights. More specifically, L1 encourages the weights to be spare, leading to better feature selection. On the other hand, L2 (also called weight decay) encourages the weights to be small, preventing them from having too much influence on the predictions.
- **Data augmentation:** This is the process of artificially increasing the size and diversity of the training dataset by applying random transformations like rotation, scaling, flipping, or cropping to the input images